دانشگاه غیرانتفاعی – صنعتی سجاد



نام و نام خانوادگی: علیرضا ریحانی یامی

شماره دانشجو: ۴۰۳۳۱۵۲۰۷

مقطع: کاردانی پیوسته

رشته: کامپیوتر نرم‌افزار (گروه :کامپیوتر)

نام درس: برنامه نویسی موبایل ۲

موضوع :بررسی بعضی متودها و کلاس ها مربوط به پایگاه داده

DBHandler.java

```
89
90      // ..........................Updating the course.............................
        1 usage
91      public void updateCourse(String originalCourseName, String courseName, String courseDescription,
92                               String courseTracks, String courseDuration) {
93
94          // calling a method to get writable database.
95          SQLiteDatabase db = this.getWritableDatabase();
96          ContentValues values = new ContentValues();
97
98          // passing all values along with its key and value pair.
99          values.put(NAME_COL, courseName);
100         values.put(DURATION_COL, courseDuration);
101         values.put(DESCRIPTION_COL, courseDescription);
102         values.put(TRACKS_COL, courseTracks);
103
104         // calling a update method to update our database and passing our values.
105         // and we are comparing it with name of our course which is stored in original name variable.
106         db.update(TABLE_NAME, values,  whereClause: "name=?", new String[]{originalCourseName});
107         db.close();
108     }
```

تحلیل کد های مربوط به updateCourse

در خط ۹۲ : یک متود با سطح دسترسی پابلیک بدون مقدار بازگشتی تعریف شده که شامل ۵ پارامتر از نوع رشته به عنوان ورودی هست.

متود getWritableDatabase

هنگامی که می‌خوایم داده‌ای را داخل دیتابیس تغییر بدهیم (مثل اضافه کردن، حذف کردن یا آپدیت کردن)، باید اول دیتابیس را در حالت قابل نوشتن باز کنیم با اجرای این دستور اگر دیتابیس وجود نداشته باشد ساخته می شود و اگر وجود داشته باشد باز میشود و میتوان عملیات های (insert,update,delete,select) را بر روی دیتابیس ایجاد کنیم. بدون فراخوانی این متود نمی توانیم چیزی ذخیره و یا حذف و آپدیت کنیم.

در خط ۹۵: یک شیء با نام dbاز کلاس SQLiteDatabase ایجاد می کنیم که میگوید متد getWritableDatabase() را از همین کلاسی که الان داخلش هستم صدا بزن و مقدارش را داخل من بریز

کلاس ContentValues

یک ظرف است که **ستون‌ها و مقدارهای جدید** را نگه می‌دارد تا SQLite بتواند آنها را در دیتابیس ذخیره یا آپدیت کند.

بدون این ظرف، اندروید اجازه نمی‌دهد به‌صورت مستقیم رشته‌ها و عددها را به دیتابیس بدهیم— باید حتماً در قالب ContentValuesبسته‌بندی شوند.

اندروید مقادیر خام را قبول نمی‌کند چون باید دقیقاً بدونه هر مقدار مال کدام ستون است ویک API امن برای خودش فراهم کند.

در خط ۹۶: یک شیء با نام values از کلاس Content Value با مقدار خالی ایجاد میکنیم.

در خط های ۹۹-۱۰۲ : می آیم مقادیری که می خواهیم در دیتایس بریزیم را درون شی values که قبل تر تعرف کردیم می ریزیم. شی values دو پارامتر به عنوان ورودی قبول می کند که باید مشخص شود؛ اولی نام ستون در دیتابیس و دیگری مقدار آن ستون است.

در خط ۱۰۶: عملیات اصلی بروزرسانی انجام میشود. در این خط متود آپدیت از شی db فراخوانی می شود با مقادیر در ابتدا نام جدول مورد نظر سپس، ستون/مقدار که باید جایگزین شوند که در values ذخیره شدند

"name=?" رشته‌ی شرط WHERE (از پارامترهای پرس‌وجو استفاده شده تا از SQL Injection جلوگیری شود). علامت سؤال (؟) با آرایه مقادیر بعدی جایگزین می‌شود.

{originalCourseName}new String[] آرایه‌ای است که مقدار جایگزین ؟ را فراهم می‌کند؛ یعنی فقط ردیف(ها)یی که ستون nameشان برابر originalCourseName است به‌روزرسانی می‌شوند.

در خط ۱۰۷: شی dbبرای آزاد سازی منابع بسته می شود. این منابع شامل موارد زیر هستند

- SQLiteبرای سرعت بیشتر یک سری داده‌ها را **کش** می‌کند. این کش توی RAM نگه داشته می‌شود. بستن دیتابیس باعث آزاد شدن این حافظه می‌شود.

- سیستم‌عامل اندروید برای هر فایل باز، یک *فایل‌هندل* نگه می‌دارد. اگر دیتابیس بسته نشود، تعداد هندل‌های باز زیاد می‌شود، سیستم‌عامل خطا می‌دهد و کل برنامه از کار می‌افتد

- وقتی دیتابیس باز است، ممکن است قفل نوشتن و قفل خواندن روی فایل دیتابیس اجرا شود. اگر بسته نشود ، این قفل‌ها ممکن است باقی بمانند و باعث شود: عملیات‌های دیگر نتوانند به دیتابیس دسترسی پیدا کنند برنامه هنگ کند

close() همه چیز را فلش (flush) می‌کند و می‌بندد و از تمام موارد ذکر شده جلو گیری میکند

جلوگیری از حذف تمام رکوردها هنگام آپگرید با روشی دیگر

به طور کلی ۴ روش دیگر وجود دارد

۱. استفاده از ALTER TABLE برای اضافه کردن ستون‌های جدید

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    for (int version = oldVersion + 1; version <= newVersion; version++) {
        switch (version) {
            case 2:

                db.execSQL("ALTER TABLE " + TABLE_NAME + " ADD COLUMN new_column TEXT");
                break;
            case 3:

                db.execSQL("ALTER TABLE " + TABLE_NAME + " ADD COLUMN another_column INTEGER");
                break;
        }
    }
}
```

در این روش، به جای حذف کل جدول، فقط تغییرات ساختاری لازم اعمال می‌شود. با استفاده از دستور ALTER TABLE ستون‌های جدید به جدول موجود اضافه می‌گردد. این روش داده‌های موجود را کاملاً حفظ می‌کند و فقط ساختار دیتابیس را گسترش می‌دهد. برای هر نسخه جدید، یک case اضافه شده و دستورات ALTER TABLE مربوط به آن نسخه اجرا می‌شود. این روش برای تغییرات کوچک مانند اضافه کردن ستون‌های جدید ایده‌آل است.

۲. ایجاد جدول موقت و انتقال داده‌ها

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion < 2) {

        db.execSQL("CREATE TABLE temp_table AS SELECT * FROM " + TABLE_NAME);


        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);


        onCreate(db);


        db.execSQL("INSERT INTO " + TABLE_NAME + " SELECT *, NULL FROM temp_table");


        db.execSQL("DROP TABLE IF EXISTS temp_table");
    }
}
```

این روش زمانی استفاده می‌شود که تغییرات ساختاری بزرگ باشند و نتوان با ALTER TABLEساده آن‌ها را انجام داد. در اینجا تمام داده‌های جدول اصلی به یک جدول موقت منتقل می‌شوند، سپس جدول اصلی حذف و مجدداً با ساختار جدید ایجاد می‌گردد. در نهایت داده‌ها از جدول موقت به جدول جدید بازگردانده شده و جدول موقت حذف می‌شود. این روش امکان تغییرات اساسی در ساختار جدول را فراهم می‌کند در حالی که داده‌ها حفظ می‌شوند.

۳. مدیریت نسخه‌به‌نسخه

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion < 2) {

        upgradeFromV1ToV2(db);
    }
    if (oldVersion < 3) {

        upgradeFromV2ToV3(db);
    }

}
```

در این روش، برای هر ارتقاء نسخه یک تابع جداگانه تعریف می‌شود. هنگام آپگرید، سیستم به ترتیب از نسخه فعلی به نسخه هدف می‌رود و برای هر پله، تابع مخصوص آن نسخه فراخوانی می‌شود. این روش بسیار منظم و قابل نگهداری است و به توسعه‌دهنده اجازه می‌دهد تغییرات هر نسخه را به طور مجزا مدیریت کند. برای برنامه‌هایی که چندین بار بروزرسانی می‌شوند، این روش امن‌ترین و قابل اطمینان‌ترین راهکار محسوب می‌شود.

٤. استفاده از کتابخانه‌هایORM

استفاده از کتابخانه‌هایی مانند:

- **Room Persistence Library**
- **GreenDAO**
- **ORMLite**

این کتابخانه‌ها مدیریت دیتابیس را ساده‌تر می‌کنند.

چرا متد deleteCourse، courseName را به عنوان ورودی میگیرد

```
public void deleteCourse(String courseName) {

    // Creating a variable to write our database.
    SQLiteDatabase db = this.getWritableDatabase();

    // Calling a method to delete our course and we are comparing it with
    db.delete(TABLE_NAME, whereClause: "name=?", new String[]{courseName})
    db.close();
}
```

در واقع در این کد گفته اید که هر جا name=?را دیدی حذفش کن و ؟ برابر هستش با courseName
پس اگر ورودی courseName را نگیرد از کجا بفهمد که چه رکوردی باید حذف شود

منابع

https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper

https://developer.android.com/reference/android/database/sqlite/SQLiteDatabase

https://developer.android.com/reference/android/content/ContentValues

https://developer.android.com/reference/android/database/sqlite/SQLiteClosable

https://learn.microsoft.com/en-us/dotnet/api/android.database.sqlite.sqlitedatabase.update

https://developer.android.com/training/data-storage/sqlite